DEVSECOPS LAB BOOK · PHASE 2: IMMUTABLE INFRASTRUCTURE

Chapter 08

OpenTofu: Infrastructure as Code

Defining all 12 lab VMs, the network, and the firewall in HCL — so the entire lab deploys with one command and rebuilds in under an hour.

`HCL · libvirt · tofu plan · tofu apply · state`

8.1 — THE PROBLEM

Why Infrastructure as Code Exists

Without IaC, infrastructure is ***snowflake*** — custom, undocumented, and hard to replicate.

Manual Approach

- "Which VMs exist?" — nobody's sure
- "What are their IPs?" — check three spreadsheets
- "What firewall rules apply?" — ssh in and look
- A manual change goes wrong — good luck finding it
- Disaster recovery means "rebuild from memory"

IaC Approach

- Read the code: exactly what exists, no guessing
- Review changes: git diff shows every edit
- Test changes: terraform plan before anything runs
- Audit history: git log — who changed what, when
- Recover fast: rebuild from code in about an hour

🏗️ *Analogy: without IaC, infrastructure is a closed-source binary. With IaC, it's open-source code — anyone can read, review, modify, and audit it.*

8.2 — TOOL CHOICE

OpenTofu vs. Terraform

Feature	Terraform	OpenTofu
Cost	Free, but licensed	Open-source (MPL 2.0)
Language	HCL	Same HCL (compatible)
Providers	HashiCorp maintained	Community maintained
State Lock	Excellent	Excellent
Security	Good	Good
Governance	Vendor-backed	Community-driven

For this lab: OpenTofu and Terraform are interchangeable. We use OpenTofu to avoid vendor lock-in.

CHAPTER GOAL

What This Chapter Delivers

Define the entire lab infrastructure — all 12 VMs, network, firewall, DNS — in OpenTofu code, instead of clicking in virt-manager or running virt-install commands.

- All 12 VMs defined in HCL
- All VMs deploy with one command
- Infrastructure is version-controlled
- Changes are reviewable (terraform plan before apply)
- Entire lab destroyed and recreated in < 1 hour
- Multiple environments (dev/staging/prod) from the same code

What is OpenTofu? *An open-source alternative to Terraform — same language, no vendor lock-in. Used identically to Terraform.*

8.3 – 8.4 — SETUP

Installation & Project Structure

Install OpenTofu (on host)

```
wget .../tofu_1.6.0_linux_amd64.zip
unzip tofu_1.6.0_linux_amd64.zip
sudo mv tofu /usr/local/bin/
tofu version
```

Expected Output

```
→ OpenTofu v1.6.0
```

Project Directory Structure

```
devsecops-lab/
├── main.tf              # Main configuration
├── variables.tf         # Variable definitions
├── outputs.tf           # Output values
├── terraform.tfvars     # Variable values
├── modules/             # Reusable modules (future)
└── environments/        # dev / staging / prod
```

8.5 — PROVIDER CONFIG

Telling OpenTofu What to Manage

The provider tells OpenTofu what infrastructure it can control — here, KVM via the libvirt provider.

1 · Install libvirt Provider

```
mkdir -p ~/.terraform.plugins/.../
  dmacvicar/libvirt/0.7.1/linux_amd64
wget <provider-release.zip>
chmod +x terraform-provider-libvirt
```

2 · providers.tf

```
required_providers {
  libvirt = {
    source = "dmacvicar/libvirt"
    version = "~> 0.7.1"
} }
provider "libvirt" {
  uri = "qemu:///system"
}
```

uri = "qemu:///system" connects OpenTofu to the local system libvirt daemon (not a remote hypervisor over SSH).

8.6 — VARIABLES.TF

Defining Network & Default VM Variables

Variable	Default	Purpose
network_name	devsecops-lab	Name of the libvirt network
network_cidr	10.0.2.0/24	Network CIDR block
network_gateway	10.0.2.1	Network gateway IP
image_path	.../ubuntu-22.04-packer.qcow2	Base image built by Packer (Ch. 07)
default_vcpu / memory / disk	2 / 2048 / 20	Overridable per-VM defaults
environment	lab	Environment name (dev/staging/prod)
domain	devsecops.com	Internal domain name

These are just defaults — every value can be overridden in terraform.tfvars.

8.6 — VARIABLES.TF

The vms Map: All 12 VMs in One Place

A single map(object) variable holds every VM's IP, vCPU, memory, and disk. Adding a 13th VM later is a five-line edit here — nothing else in the codebase changes.

```
variable "vms" {
  type = map(object({
    ip = string, vcpu = number, memory = number, disk_gb = number
  }))
  default = {
    "sec-dns-01" = { ip = "10.0.2.10", vcpu = 2, memory = 2048, disk_gb = 20 }
    "sec-vcs-01" = { ip = "10.0.2.14", vcpu = 8, memory = 8192, disk_gb = 80 }
    # ... 10 more VMs
  }
}
```

12 VMs defined, from sec-dns-01 (2 vCPU) to sec-registry-01 (100GB disk)

1 map = 1 source of truth — no drift between docs and reality

8.7 - 8.8

network.tf & vms.tf — The Core Resources

network.tf

- resource libvirt_network "devsecops"
- mode = bridge, bridge = virbr-lab
- addresses = [var.network_cidr]
- autostart = true
- DHCP disabled — every VM gets a static IP from the vms map

vms.tf — three resources, one for_each

`libvirt_volume.vm_disk`

one qcow2 disk per VM, sized from disk_gb

`libvirt_network_interface.vm_net`

static IP + generated MAC + hostname

`libvirt_domain.vm`

the actual VM: memory, vcpu, disk, NIC, console

cpu { mode = "host-passthrough" } → uses the host's real CPU features for best performance.

for_each = var.vms means one block of HCL creates all 12 VMs.

8.9 - 8.10

Outputs & terraform.tfvars

outputs.tf

```
output "network_id" { ... }

output "vms" {
  # name, ip, vcpu, memory, disk_gb
  # for every deployed VM
}

output "ssh_commands" {
  # ssh ubuntu@<ip> per VM
}
```

terraform.tfvars

```
environment = "lab"
domain      = "devsecops.com"

# Keep default VMs as-is
# Or override here if needed:
# vms = { ... }
```

This is the file you edit for day-to-day changes — variables.tf holds structure, tfvars holds values.

8.11 - 8.13

Init → Plan → Apply

1

`tofu init`

Downloads providers, initializes the workspace

"Terraform has been successfully initialized!"

2

`tofu plan -out=tfplan`

Shows exactly what will be created/changed/destroyed

"Plan: 12 to add, 0 to change, 0 to destroy."

3

`tofu apply tfplan`

Executes the plan — 2–5 min, VMs boot in parallel

"Apply complete! Resources: 12 added."

Never skip plan — it's your only preview before OpenTofu touches real infrastructure.

VERIFICATION

Confirming All 12 VMs Are Running

```
$ virsh list --all

 Id   Name                 State
 ------------------------------------
 1    sec-dns-01           running
 2    sec-idm-01           running
 3    sec-edge-01          running
 ...  (12 total)                 running
```

Result: 12 VMs, one command, zero clicks in virt-manager. The exact same result every time.

8.14 - 8.15

Multiple Environments & Safe Changes

Environments via tfvars

```
environments/dev.tfvars
  network_cidr = "10.0.2.0/24"

environments/prod.tfvars
  network_cidr = "10.1.0.0/24"

tofu apply -var-file=environments/dev.tfvars
```

Adding / Resizing a VM

1. Edit variables.tf (add VM or change memory/vcpu)
2. tofu plan
   → shows exactly what changes
3. tofu apply
   (resize may need a reboot)

Example plan output — resizing sec-vcs-01

```
libvirt_domain.vm["sec-vcs-01"] will be updated in-place:
  ~ memory = 8192 -> 16384
```

This is IaC's superpower: every change is previewed, reviewed, and reversible.

8.16 - 8.17

Destroying Infrastructure & Managing State

Destroy — Use With Caution

```
tofu plan -destroy   # preview
tofu destroy          # deletes ALL VMs + disks + network
```

This deletes all VMs and data — no undo.

State Commands

```
tofu show                 # view state
tofu state list            # list resources
tofu state show <resource>
tofu state mv <old> <new>
tofu state rm <resource>
```

Golden rule: never hand-edit terraform.tfstate. Always use tofu commands — direct edits corrupt state tracking.

terraform.tfstate tracks the real-world mapping between your HCL code and actual running resources.

8.18 — VERSION CONTROL

Storing Infrastructure in Git

Commands

```
git init
git add .gitignore
git add devsecops-lab/
git commit -m "Initial lab infra as code"
git tag -a v1.0 -m "First stable deployment"
```

.gitignore — never commit these

- *.tfstate* — contains real IPs, IDs, secrets
- .terraform/ — local provider cache
- .terraform.lock.hcl — regenerated by init
- *.tfvars (except example.tfvars) — may hold secrets
- crash.log — debug artifacts

State files contain real IPs and resource IDs — treat them like secrets, never commit them.

8.19 — VALIDATION

validate-chapter-08.sh

- OpenTofu installed — tofu version returns a result
- Config files exist — variables.tf, providers.tf, vms.tf, outputs.tf, terraform.tfvars
- Configuration is valid — tofu validate passes
- State file exists and VM count is tracked
- All 12 lab VMs are running (virsh list --name)
- Pass/Fail counters printed, exit 1 on any failure

```
Run it any time: bash ~/lab/scripts/validate-chapter-08.sh
```

8.20 — TROUBLESHOOTING

Common Errors & Fixes

"No such file or directory" for image

Packer image doesn't exist yet — build it first (Chapter 07), then check /var/lib/libvirt/images/

"Plugin error: permission denied"

libvirt provider can't reach KVM — sudo usermod -aG libvirt $USER && newgrp libvirt, or sudo tofu apply

"Error creating domain: Name already exists"

VM already exists (e.g. from Chapter 01) — virsh destroy + virsh undefine + remove the old qcow2, then re-apply

Apply hangs or takes very long

Normal — 12 VMs starting in parallel takes 5–10 minutes. Watch progress with virsh list --all

8.21 — SUMMARY

Chapter Summary & Key Takeaway

What We Built	Purpose
OpenTofu configuration	Infrastructure defined as code
Providers & network	KVM/libvirt setup in HCL
VM definitions	All 12 VMs in one variables map
Deployment automation	One-command lab creation
State management	Track infrastructure changes safely
Version control	Git-based infrastructure audit trail

Infrastructure becomes repeatable: identical provisioning every time, reviewable changes, straightforward rollback (git revert), and disaster recovery in under an hour.

NEXT CHAPTER

Chapter 09 — Ansible: Configuration Management

OpenTofu creates the VMs. **Ansible configures what runs on them.**

- Install services (BIND9, LLDAP, Authentik, etc.)
- Deploy configuration files
- Start services
- Idempotent — safe to run multiple times
- Roll back configurations easily

After Chapter 09, the entire Phase 1 foundation (Chapters 01–06) deploys automatically from code.